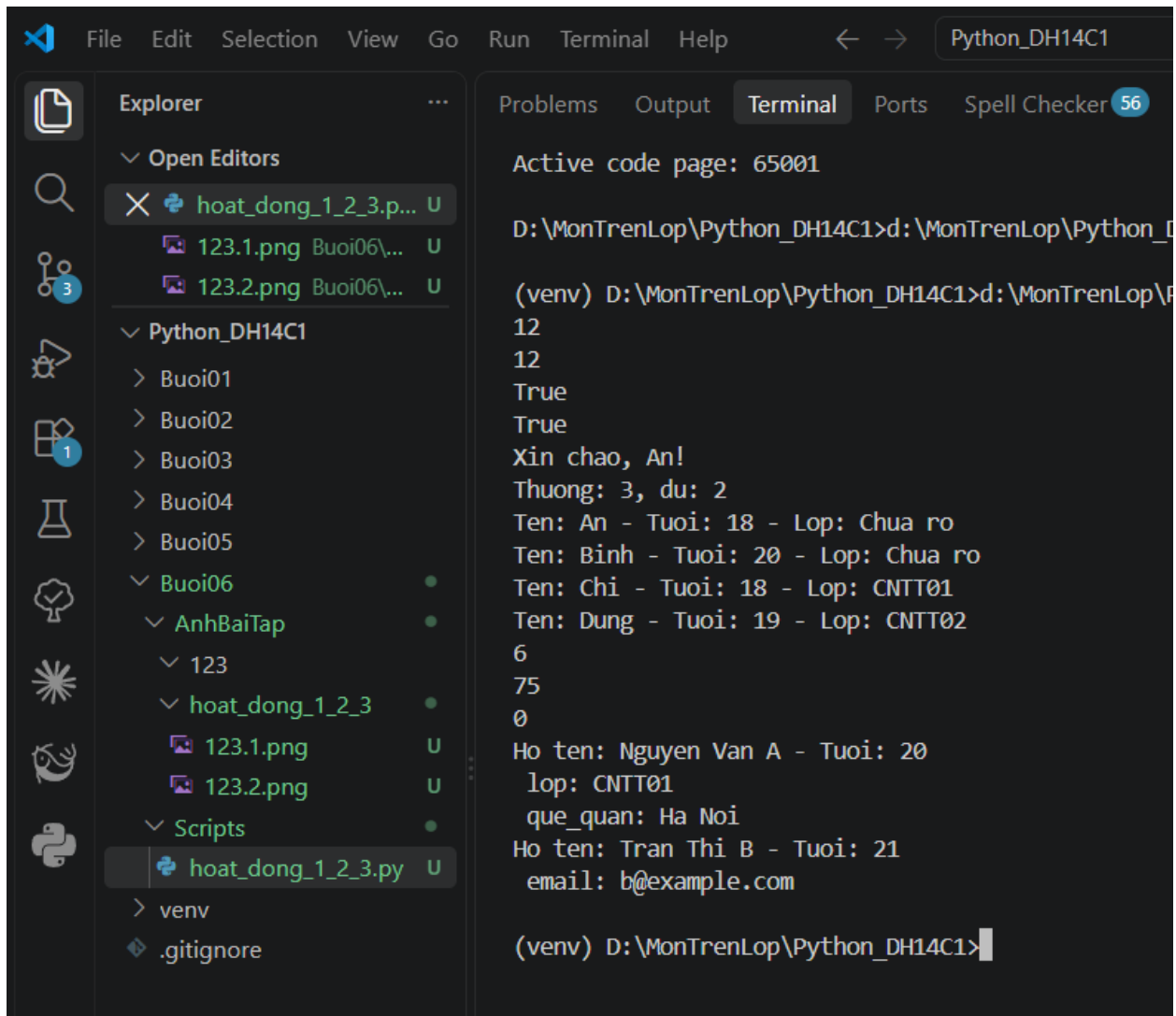
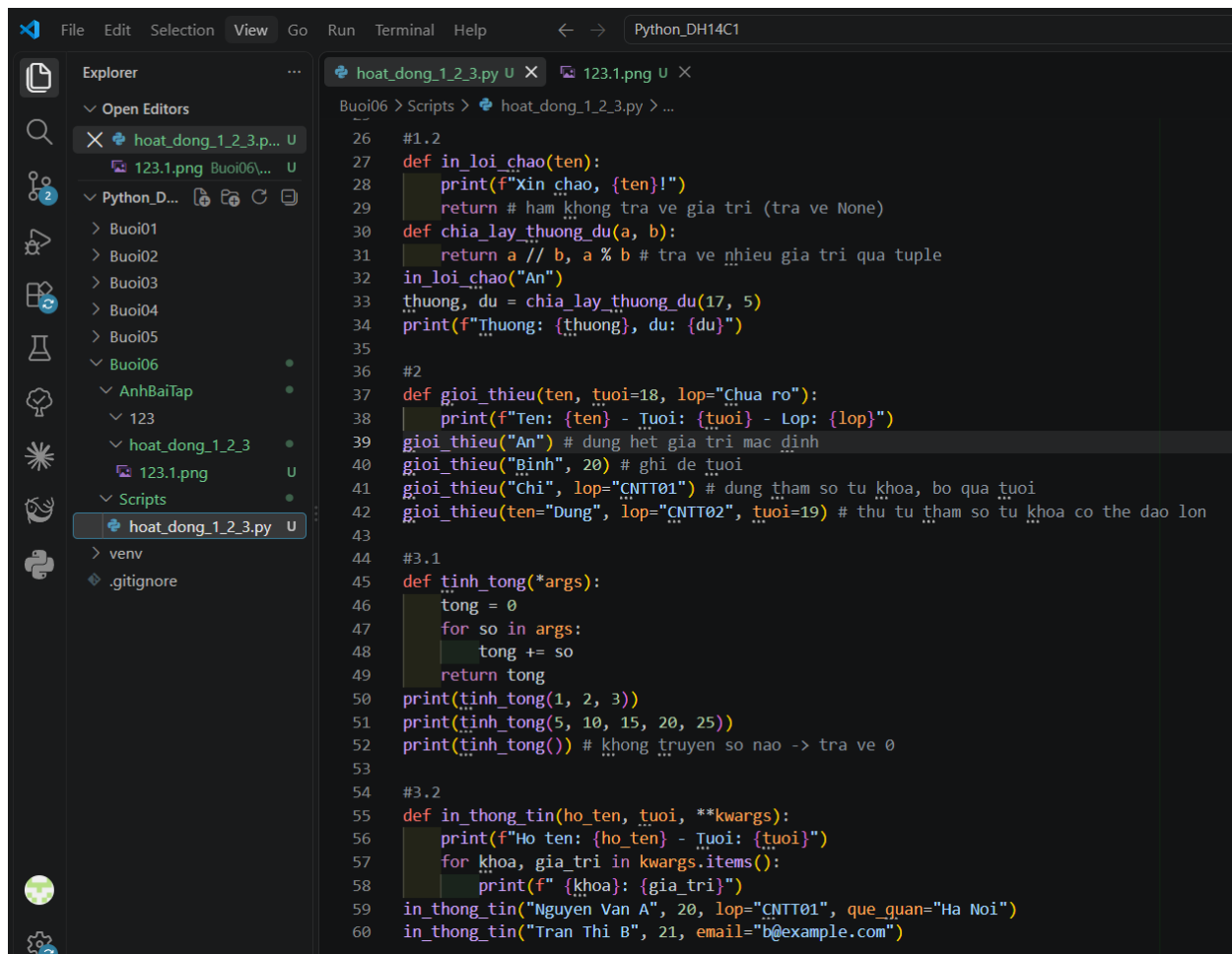


BÁO CÁO

hd1.2.3



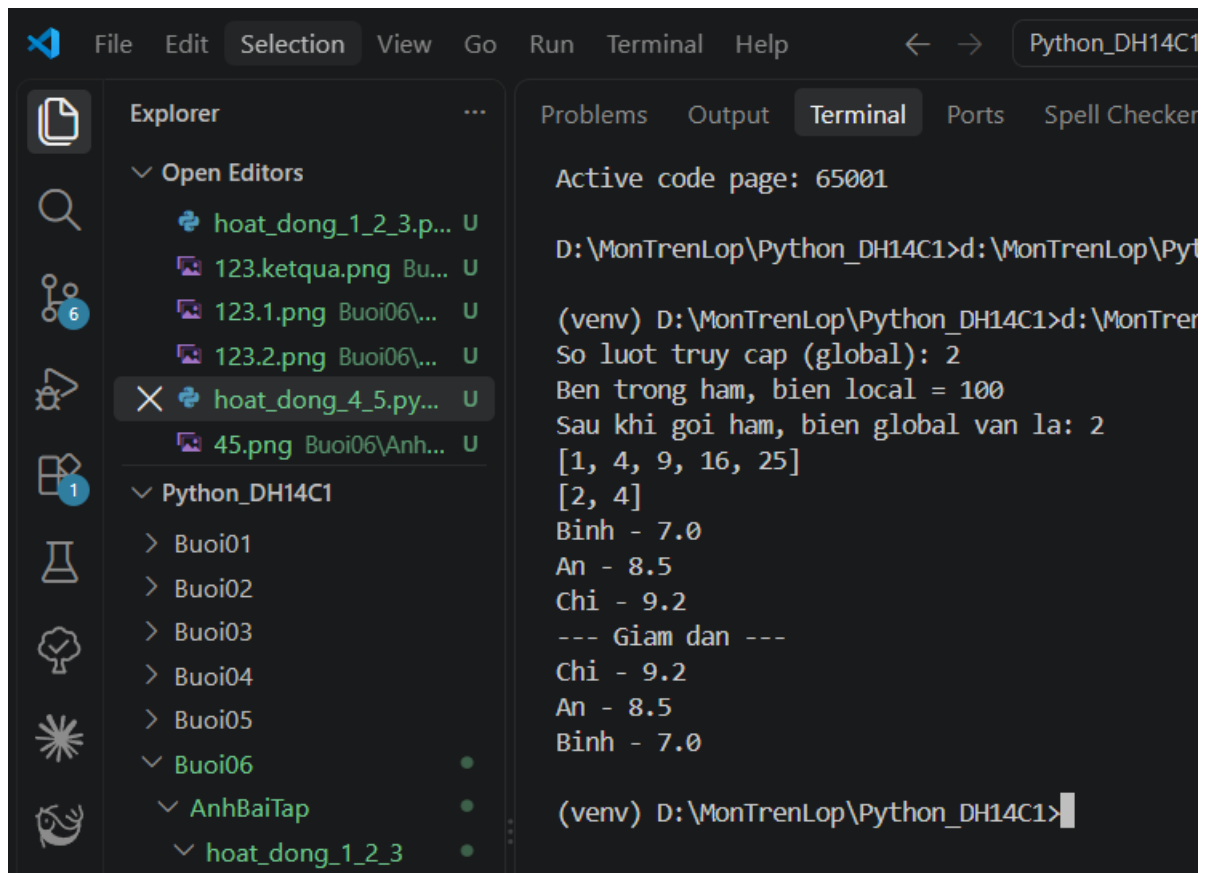


```
26 #1.2
27 def in_loi_chao(ten):
28     print(f"xin chào, {ten}!")
29     return # ham không trả về giá trị (trả về None)
30 def chia_lay_thuong_du(a, b):
31     return a // b, a % b # trả về nhiều giá trị qua tuple
32 in_loi_chao("An")
33 thuong, du = chia_lay_thuong_du(17, 5)
34 print(f"Thuong: {thuong}, du: {du}")
35
36 #2
37 def gioi_thieu(ten, tuoi=18, lop="Chưa rõ"):
38     print(f"Ten: {ten} - Tuổi: {tuoi} - Lop: {lop}")
39 gioi_thieu("An") # dùng hết giá trị mặc định
40 gioi_thieu("Bình", 20) # ghi đè tuổi
41 gioi_thieu("Chi", lop="CNTT01") # dùng tham số từ khóa, bỏ qua tuổi
42 gioi_thieu(ten="Dung", lop="CNTT02", tuoi=19) # thu từ tham số từ khóa có thể đảo lộn
43
44 #3.1
45 def tinh_tong(*args):
46     tong = 0
47     for so in args:
48         tong += so
49     return tong
50 print(tinh_tong(1, 2, 3))
51 print(tinh_tong(5, 10, 15, 20, 25))
52 print(tinh_tong()) # không truyền số nào -> trả về 0
53
54 #3.2
55 def in_thong_tin(ho_ten, tuoi, **kwargs):
56     print(f"Ho ten: {ho_ten} - Tuổi: {tuoi}")
57     for khoa, gia_tri in kwargs.items():
58         print(f" {khoa}: {gia_tri}")
59 in_thong_tin("Nguyen Van A", 20, lop="CNTT01", que_guan="Hà Nội")
60 in_thong_tin("Tran Thi B", 21, email="b@example.com")
```

The image shows a Visual Studio Code editor window with the file explorer on the left and a Python script open in the main editor. The file explorer shows a project structure with folders 'Buoi01' through 'Buoi06', a subfolder 'AnhBaiTap' containing '123', and a 'Scripts' folder containing the active file 'hoat_dong_1_2_3.py'. The script itself contains the following Python code:

```
1  #1.1
2  def uscln(a, b):
3      while b != 0:
4          a, b = b, a % b
5      return a
6  def bscnn(a, b):
7      return a * b // uscln(a, b)
8  def kiem_tra_nguyen_to(n):
9      if n < 2:
10         return False
11         for i in range(2, n):
12             if n % i == 0:
13                 return False
14         return True
15  def kiem_tra_so_hoan_thien(n):
16     tong_uoc = 0
17     for i in range(1, n):
18         if n % i == 0:
19             tong_uoc += i
20     return tong_uoc == n
21  print(uscln(24, 36))
22  print(bscnn(4, 6))
23  print(kiem_tra_nguyen_to(29))
24  print(kiem_tra_so_hoan_thien(28)) # 28 = 1 + 2 + 4 + 7 + 14
25
```

hd45



```
1 #4
2 so_luot_truy_cap = 0 # bien global
3 def tang_luot_truy_cap():
4     global so_luot_truy_cap
5     so_luot_truy_cap += 1
6 def vi_du_bien_local():
7     so_luot_truy_cap = 100 # day la bien LOCAL, khac voi bien global cung ten
8     print("Ben trong ham, bien local =", so_luot_truy_cap)
9 tang_luot_truy_cap()
10 tang_luot_truy_cap()
11 print("So luot truy cap (global):", so_luot_truy_cap)
12 vi_du_bien_local()
13 print("Sau khi goi ham, bien global van la:", so_luot_truy_cap)]
14
15 #5.1
16 danh_sach_so = [1, 2, 3, 4, 5]
17 binh_phuong = list(map(lambda x: x ** 2, danh_sach_so))
18 print(binh_phuong)
19
20 #5.2
21 so_chan = list(filter(lambda x: x % 2 == 0, danh_sach_so))
22 print(so_chan)
23
24 #5.3
25 danh_sach_sv = [
26     {"ten": "An", "diem": 8.5},
27     {"ten": "Binh", "diem": 7.0},
28     {"ten": "Chi", "diem": 9.2},
29 ]
30 sap_xep_theo_diem = sorted(danh_sach_sv, key=lambda sv: sv["diem"])
31 sap_xep_giam_dan = sorted(danh_sach_sv, key=lambda sv: sv["diem"], reverse=True)
32 for sv in sap_xep_theo_diem:
33     print(sv["ten"], "-", sv["diem"])
34 print("--- giam dan ---")
35 for sv in sap_xep_giam_dan:
36     print(sv["ten"], "-", sv["diem"])
```

The screenshot shows the Visual Studio Code editor with a Python file named `hoat_dong_6.py` open. The file contains two functions: `giai_thua_de_quy(n)` and `giai_thua_lap(n)`. The `giai_thua_de_quy` function uses a recursive approach to calculate the nth Fibonacci number, while the `giai_thua_lap` function uses an iterative approach. The code is as follows:

```
1 #6.1
2 def giai_thua_de_quy(n):
3     if n <= 1: # dieu kien dung
4         return 1
5     return n * giai_thua_de_quy(n - 1)
6 def giai_thua_lap(n):
7     ket_qua = 1
8     for i in range(1, n + 1):
9         ket_qua *= i
10    return ket_qua
11 print(giai_thua_de_quy(5), "-", giai_thua_lap(5))
12
13 #6.2
14 def fibonacci_de_quy(n):
15     if n <= 1: # dieu kien dung
16         return n
17     return fibonacci_de_quy(n - 1) + fibonacci_de_quy(n - 2)
18 for i in range(10):
19     print(fibonacci_de_quy(i), end=" ")
20 print()
```

The screenshot shows the Visual Studio Code editor with the same Python file `hoat_dong_6.py` open. The terminal window is active, showing the output of the script. The output is as follows:

```
Active code page: 65001
D:\MonTrenLop\Python_DH14C1>d:\MonTrenLop
(venv) D:\MonTrenLop\Python_DH14C1>d:\MonTrenLop
120 - 120
0 1 1 2 3 5 8 13 21 34
(venv) D:\MonTrenLop\Python_DH14C1>
```

hd7

This screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows a project named 'Python_DH14C1' with a folder 'Bui06' containing subfolders 'Bui01' through 'Bui05'. The 'Open Editors' list shows several files, including 'utils.py' which is currently selected. The code editor displays the contents of 'utils.py', which defines several utility functions for string manipulation and palindrome checking.

```
1 def dao_nguoc_chuoi(chuoi):
2     return chuoi[::-1]
3 def kiem_tra_palindrome(chuoi):
4     return chuoi == chuoi[::-1]
5 def chuan_hoa_ho_ten(chuoi):
6     return " ".join(chuoi.split()).title()
7 def uscln(a, b):
8     while b != 0:
9         a, b = b, a % b
10    return a
11 def kiem_tra_nguyen_to(n):
12     if n < 2:
13         return False
14     for i in range(2, n):
15         if n % i == 0:
16             return False
17     return True
```

This screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the same project structure as the previous screenshot. The 'Open Editors' list shows several files, including 'main.py' which is currently selected. The code editor displays the contents of 'main.py', which imports the 'utils' module and calls several of its functions to demonstrate their usage.

```
1 import utils
2 print(utils.dao_nguoc_chuoi("Python"))
3 print(utils.kiem_tra_palindrome("madam"))
4 print(utils.chuan_hoa_ho_ten(" nguyen van an "))
5 print(utils.uscln(24, 36))
6 print(utils.kiem_tra_nguyen_to(29))
```

This screenshot shows the Visual Studio Code editor with the file explorer on the left and the terminal on the right. The file explorer shows the same project structure. The 'Open Editors' list shows several files, including 'main.py' which is currently selected. The terminal shows the output of running the 'main.py' file, displaying the results of the function calls.

```
D:\MonTrenLop\Python_DH14C1>d:\MonTrenLop\Python_DH14C1\venv\Scripts\activate.bat
(venv) D:\MonTrenLop\Python_DH14C1>d:\MonTrenLop\Python_DH14C1\venv\Scripts\python.
nohtyP
True
Nguyen Van An
12
True
(venv) D:\MonTrenLop\Python_DH14C1>
```

Câu Hỏi

hd2

Khi gọi hàm bằng tham số từ khóa (ten=..., lop=...), thứ tự truyền vào không quan trọng là vì:

- Xác định theo tên, không theo vị trí: Python sẽ dựa trực tiếp vào tên tham số để gán giá trị tương ứng vào đúng biến trong hàm, thay vì phụ thuộc vào thứ tự đứng trước hay đứng sau như khi gọi theo vị trí.

hd4

Chương trình sẽ báo lỗi UnboundLocalError vì:

- Python coi đó là biến local: Khi trong hàm có câu lệnh gán `so_luot_truy_cap += 1` (tương đương `so_luot_truy_cap = so_luot_truy_cap + 1`), Python tự động hiểu `so_luot_truy_cap` là một biến cục bộ (local variable) của hàm đó.
- Đọc giá trị trước khi khởi tạo: Khi thực hiện về phải (`so_luot_truy_cap + 1`), Python cố gắng đọc giá trị của biến local này, nhưng lúc này biến local chưa hề được gán giá trị ban đầu nào bên trong hàm dẫn đến lỗi UnboundLocalError.

hd5.3

Dùng `key=lambda` linh hoạt hơn cách đặt điểm trước tên trong tuple vì:

- Không cần sửa đổi dữ liệu gốc: Dùng `key=lambda` cho phép chỉ định trực tiếp thuộc tính cần sắp xếp (như `sv["diem"]`) mà không cần tốn công tạo lại cấu trúc dữ liệu mới hay chuyển đổi dữ liệu thành dạng tuple (diem, ten).
- Dễ dàng thay đổi tiêu chí sắp xếp: Có thể tùy biến tiêu chí sắp xếp theo bất kỳ thuộc tính nào (theo tên, theo điểm, theo độ dài tên,...) chỉ bằng cách đổi biểu thức sau `lambda`, thay vì phải sắp đặt lại thứ tự các phần tử trong tuple.

hd6.1

So sánh việc tính giai thừa bằng vòng lặp và đệ quy:

- Tối ưu bộ nhớ & hiệu năng: Vòng lặp (giai_thua_lap) tối ưu hơn vì chạy trực tiếp trong bộ nhớ cố định. Trong khi đó, đệ quy (giai_thua_de_quy) phải lưu lại các khung hàm cho mỗi lần gọi đệ quy, tốn bộ nhớ hơn.
- Tránh lỗi tràn bộ nhớ: Vòng lặp có thể xử lý số lượng bước lặp rất lớn mà không sợ lỗi, còn đệ quy nếu `n` quá lớn sẽ gây ra lỗi tràn bộ nhớ đệ quy (RecursionError).

hd6.2

Đệ quy Fibonacci "tốn kém" và chạy chậm hơn hẳn so với tính giai thừa là vì:

- Tính toán lặp lại nhiều lần: Để tính $F(n)$, thuật toán phải chia nhỏ thành 2 cuộc gọi đệ quy $F(n-1)$ và $F(n-2)$. Cả hai nhánh này lại tiếp tục tính lại các giá trị giống hệt nhau (ví dụ: $F(3)$, $F(2)$ bị lặp lại rất nhiều lần).
- Số lần gọi hàm tăng theo cấp số nhân: Với Fibonacci, độ phức tạp thời gian là 2^n (số lần gọi hàm nhân đôi ở mỗi tầng), trong khi giai thừa chỉ là $O(n)$ (chỉ gọi đệ quy đúng n lần).

hd7

Hai file `utils.py` và `main.py` cần đặt trong cùng một thư mục để lệnh `import utils` hoạt động đúng vì:

- Tự động tìm kiếm ở thư mục hiện tại: Khi chạy `python main.py`, Python sẽ tự động thêm thư mục chứa `main.py` (thư mục hiện tại) vào danh sách đường dẫn tìm kiếm module (`sys.path`).
- Định vị chính xác module: Việc đặt chung thư mục giúp trình biên dịch Python ngay lập tức tìm thấy file `utils.py` mà không bị báo lỗi `ModuleNotFoundError`.